

HERO PHOTO : the chassis on the workbench, 3:2 landscape

Retriever

Open source autonomous outdoor mobile robot

[Sponsorship dossier](#) · [Technical project brief](#)

William Hanczyk
KEDGE Business School
Bordeaux, France

[YOUR EMAIL]
[github.com/\[YOUR HANDLE\]/retriever](https://github.com/[YOUR HANDLE]/retriever)

August 2026
Version 1.0

Hardware and software released under
an open source licence.

CAD render

3D printed parts

Aluminium frame

Project overview

Retriever is a four wheel drive outdoor mobile robot, built independently and released as open hardware. Its single objective is autonomous outdoor navigation using ROS 2 Jazzy and the Nav2 stack, on a chassis that anyone can reproduce from documented, commodity parts.

The project is an open rebuild inspired by the **Clearpath Husky**, a research platform that has become a reference in mobile robotics laboratories. The Husky is an excellent machine and a commercial product priced accordingly. Retriever asks a different question: how much of that capability can be reached by one person, from recovered and off the shelf components, if the engineering is done properly rather than cheaply? The mechanical design was modelled from scratch, and the electrical and software architecture was designed independently.

What makes the project unusual is not the hardware, which is deliberately ordinary. It is the discipline applied to it: a complete architecture dossier written before the first part was cut, a documented failure mode analysis, a hardware safety chain that no software can override, and a build plan with a measurable exit criterion at every stage.

PHOTO : The chassis under assembly

Figure 1: The chassis under assembly

Technical specification

Configuration	4WD skid steer, outdoor
Structure	Aluminium extrusion frame, laser cut panels, 3D printed mounts
Target mass	35 kg in running order
Target speed	1.5 m/s
Drivetrain	4 × 6.5 inch hub motors, 250 W each
Motor drivers	4 × BLDC controllers, 36 to 48 V
Battery	36 V, 280 Wh lithium ion, 10S3P
Endurance	70 min at 200 W average
Compute	x86 SBC, Ubuntu 24.04, ROS 2 Jazzy
Real time	3 × ESP32 on CAN 2.0A, 500 kbit/s
Localisation	GNSS, IMU and wheel odometry, dual EKF
Navigation	Nav2, collision monitor, twist mux
Licence	Open hardware and software

System architecture

Three decisions define the platform.

A CAN bus, not USB, carries every command. Four BLDC controllers chopping tens of amps within 30 cm of the signal wiring make USB the wrong choice for the control path: it is a master slave bus, dynamically enumerated, with no frame priority. CAN 2.0A at 500 kbit/s links the computer to every microcontroller instead. USB is retained only for high bandwidth perception sensors, whose loss degrades the mission but cannot cause dangerous motion.

The main computer never does real time. It runs perception, localisation, Nav2 and the mission state machine, and it produces **setpoints**. A dedicated safety microcontroller decides whether those setpoints may be executed. The computer requests arming; the safety controller grants or refuses it. A ROS 2 bug, an out of memory kill or a blocked kernel cannot produce movement.

Safety is hardware, and independent of software. Six stop levels, from a 100 ms software collision monitor down to a mechanically latching emergency stop wired in series with the main DC contactor coil. The last two levels contain no code at all. No software, no microcontroller and no logic supply can hold that contactor closed once the mushroom button is pressed.

PHOTO : System block diagram

Figure 2: System block diagram

Where the project stands

Design is complete. The full architecture dossier runs to eight documents covering electrical design, communications, ROS 2 architecture, safety, network, code organisation, test strategy and a costed bill of materials. The mechanical design is finished in CAD.

Construction has started. The structural parts are cut and the frame is going together: an aluminium extrusion chassis, laser cut panels, and 3D printed mounts for the electronics, sensors and battery.

Stage	Exit criterion	State
1	Architecture and CAD design complete	done
2	Chassis fabricated and assembled	current
3	Power chain energised, no motor connected	next
4	Motors driven, wheels raised, all stop tests pass	planned
5	Autonomous outdoor navigation to a waypoint	planned

Stage 5 is where the hardware described below is needed, and it is the stage this project cannot reach on a student budget alone.

Beyond stage 5, the intention is to mount my Niryo One arm on Retriever and turn the platform into a mobile manipulator: a robot that navigates to a point and then does something once it arrives. The architecture was designed for this from the start. Adding a sixth node to the CAN bus takes two wires and a busbar tap, not a rewire.

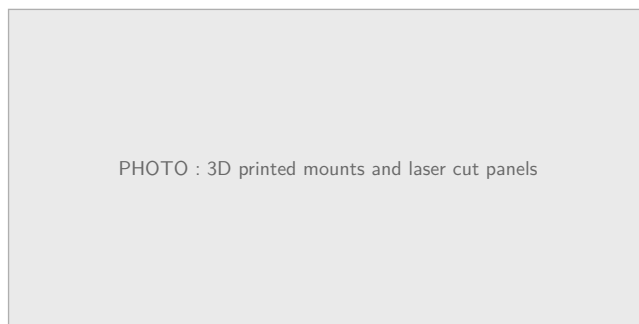


Figure 3: 3D printed mounts and laser cut panels

Looking for sponsors

I am looking for partners to help acquire the three items below. Each is described by its minimum specification rather than by a product reference, so that a sponsor can propose whatever suits them best. Support in kind, a discount, or a straightforward contribution are all equally welcome.

1. Printed circuit boards

Three custom boards are currently being designed in EasyEDA: a motor driver interface, a safety and power management board, and a CAN distribution board. The project needs them fabricated as 2 layer and 4 layer prototypes. All files will be released openly.

2. Embedded AI compute

The current x86 board runs the navigation stack but cannot support real time inference for outdoor perception. The project needs one embedded AI development kit of the 8 GB class, to handle visual obstacle classification and terrain segmentation.

3. Outdoor 3D LiDAR

This is the blocking item. The 2D LiDAR currently on hand is specified for indoor use only: its datasheet gives a 0 to 2000 lux test envelope and no IP rating, while direct sunlight reaches 100 000 lux and saturates the receiver. Outdoor autonomy is not achievable with it.

The project needs one 3D LiDAR with 30 m range or better, a 360° horizontal field of view, specified sunlight immunity, and an available ROS 2 driver.

What a sponsor receives

Retriever is a documentation project as much as an engineering one, and the documentation is where a sponsor gets value.

Integration	A written and filmed integration tutorial for the sponsored part, published openly. The kind of content that is hard to produce in house and that answers the questions real users ask.
Field test	A field test video of the robot operating outdoors with the sponsored hardware, including honest performance data.
Attribution	Logo on the chassis, in the repository README and on the project page.
Feedback	Honest feedback on how the part behaves in real use, on the robot rather than on a desk. Shared privately first.

About

I am William Hanczyk, a business student at KEDGE Business School in Bordeaux. I have been building robots since childhood, well before I studied anything else. My previous project was a full rebuild of the Niryo One, an open source six axis robotic arm, which taught me most of what I know about motion control, mechanical tolerances and the gap between a CAD model and a machine that actually works.

Retriever is the logical next step, and a considerably harder one: an outdoor platform carrying its own power, its own safety chain and its own autonomy. I build it outside any academic programme, on my own time and my own budget.

The project is deliberately open. Its purpose is to show that a rigorous, safety first mobile robot can be built and documented outside a laboratory, and to leave behind something the next person can reproduce.

Contact William Hanczyk · [YOUR EMAIL]
github.com/[YOUR HANDLE]/retriever

Inspired by the Clearpath Husky platform. Mechanical design modelled from scratch; some ROS 2 package conventions follow the open source *husky/husky* repository (BSD-3-Clause). Clearpath Robotics is not affiliated with this project.